Design, develop, code and run the program in any suitable language to solve the commission problem. Analyze it from the perspective of boundary value testing, derive different test cases, execute these test cases and discuss the test results

A

Commission Problem

The **Commission Problem** is a software testing problem used to demonstrate decision making, conditional logic, and testing techniques such as **Boundary Value Analysis (BVA)** and **Equivalence Class Partitioning (ECP)**.

Problem Statement

A company sells three products:

Product	Unit Price (₹)
Lock	45
Stock	30
Barrel	25

A salesperson enters the number of units sold for each product. The program calculates:

1. **Total Sales Amount**
2. **Commission Earned**

The commission is based on the following slabs:

Total Sales (₹)	Commission Rate
Up to 1000	10%
1001–1800	15% on the amount above ₹1000
Above 1800	20% on the amount above ₹1800, after applying the previous slabs

Algorithm

1. Read the number of locks, stocks, and barrels sold.
2. Validate that all quantities are non-negative.
3. Calculate the total sales using:

$$\text{Sales} = (45 \times \text{Locks}) + (30 \times \text{Stocks}) + (25 \times \text{Barrels})$$

4. Determine the commission:
    - If sales ≤ ₹1000:
        - Commission = 10% of sales.
    - If ₹1000 < sales ≤ ₹1800:
        - Commission = ₹100 + 15% of (sales − ₹1000).
    - If sales > ₹1800:
        - Commission = ₹220 + 20% of (sales − ₹1800).
5. Display the total sales and commission.

Boundary Value Analysis (BVA)

Definition

Boundary Value Analysis (BVA) is a black-box software testing technique that focuses on testing values at the **edges (boundaries)** of valid input ranges. Many software defects occur at boundary conditions because programmers often make mistakes when implementing comparison operators such as <, <=, >, or >=.

Instead of testing every possible input, BVA tests values:

- Just below the boundary
- Exactly at the boundary
- Just above the boundary

This helps detect off-by-one and boundary-related errors efficiently.

Applying BVA to the Commission Problem

The commission calculation changes at two critical sales values:

- **₹1000**
- **₹1800**

These are the decision boundaries.

Boundary 1: ₹1000

Test the values:

Test Value	Reason
₹999	Just below the boundary

Test Value	Reason
₹1000	At the boundary
₹1001	Just above the boundary

Boundary 2: ₹1800

Test the values:

Test Value	Reason
₹1799	Just below the boundary
₹1800	At the boundary
₹1801	Just above the boundary

BVA Test Cases

Test Case	Sales (₹)	Boundary Type	Expected Result
TC1	999	Just below 1000	10% commission
TC2	1000	At 1000	10% commission
TC3	1001	Just above 1000	15% slab begins
TC4	1799	Just below 1800	15% commission
TC5	1800	At 1800	End of 15% slab
TC6	1801	Just above 1800	20% slab begins
TC7	0	Lower boundary	Zero commission
TC8	Negative quantity	Invalid input	Error message

Advantages of Boundary Value Analysis

- Detects errors occurring at boundary conditions.
- Reduces the number of test cases while maintaining good test coverage.
- Improves software reliability by ensuring correct behavior at critical input values.

C Program

```
#include <stdio.h>
```

```
int main()
{
    int lock, stock, barrel;
    float sales, commission;

    printf("Enter number of Locks sold: ");
    scanf("%d", &lock);

    printf("Enter number of Stocks sold: ");
    scanf("%d", &stock);

    printf("Enter number of Barrels sold: ");
    scanf("%d", &barrel);

    if(lock < 0 || stock < 0 || barrel < 0)
    {
        printf("Invalid Input! Quantity cannot be negative.\n");
        return 0;
    }

    sales = (lock * 45) + (stock * 30) + (barrel * 25);

    if(sales <= 1000)
    {
        commission = sales * 0.10;
    }
    else if(sales <= 1800)
    {
        commission = (1000 * 0.10) + ((sales - 1000) * 0.15);
    }
    else
    {
        commission = (1000 * 0.10) +
                     (800 * 0.15) +
                     ((sales - 1800) * 0.20);
    }

    printf("\nTotal Sales = %.2f", sales);
    printf("\nCommission = %.2f\n", commission);

    return 0;
}
```

A salesperson sells three products:

Product	Price (₹)
Lock	45
Stock	30
Barrel	25

Test Case 1

```
Enter number of Locks sold: 10
```

```
Enter number of Stocks sold: 10
Enter number of Barrels sold: 9

Total Sales = 975.00
Commission = 97.50
```

Test Case 3

```
Enter number of Locks sold: 30
Enter number of Stocks sold: 15
Enter number of Barrels sold: 5

Total Sales = 1925.00
Commission = 245.00
```

Test Case	Sales (₹)	Expected Commission (₹)	Actual Commission (₹)	Result
TC1	999	99.90	99.90	Pass
TC2	1000	100.00	100.00	Pass
TC3	1001	100.15	100.15	Pass
TC4	1799	219.85	219.85	Pass
TC5	1800	220.00	220.00	Pass
TC6	1801	220.20	220.20	Pass
TC7	0	0.00	0.00	Pass
TC8	-10 (Invalid Input)	Error Message	Error Message	Pass

Design, develop, code and run the program in any suitable language to implement the NextDate function. Analyze it from the perspective of boundary value testing, derive different test cases, execute these test cases and discuss the test results

Develop a program that accepts a valid date (day, month, and year) and outputs the **next immediate date**.

Examples:

- Input: **15/08/2026**
- Output: **16/08/2026**
- Input: **31/12/2026**
- Output: **01/01/2027**

Algorithm

1. Read day, month, and year.
2. Check whether the year is a leap year.
3. Determine the maximum number of days in the given month.
4. If the day is less than the maximum days, increment the day.
5. Otherwise:
    - Set day = 1.
    - Increment the month.
6. If the month exceeds 12:
    - Set month = 1.
    - Increment the year.
7. Display the next date.

C Program

```
#include<stdio.h>

int main()
{
    int day, month, year;
    int maxDays;

    printf("Enter day month year: ");
    scanf("%d%d%d",&day,&month,&year);

    // Leap year check
    if((year%400==0) || (year%4==0 && year%100!=0))
        maxDays=29;
    else
        maxDays=28;

    switch(month)
    {
        case 1:
        case 3:
        case 5:
        case 7:
        case 8:
        case 10:
        case 12:
            maxDays=31;
            break;

        case 4:
        case 6:
        case 9:
        case 11:
            maxDays=30;
            break;

        case 2:
```

```
            // maxDays already decided (28/29)
            break;

        default:
            printf("Invalid Month");
            return 0;
    }

    if(day<1 || day>maxDays)
    {
        printf("Invalid Date");
        return 0;
    }

    day++;

    if(day>maxDays)
    {
        day=1;
        month++;

        if(month>12)
        {
            month=1;
            year++;
        }
    }

    printf("Next Date = %02d/%02d/%04d",day,month,year);

    return 0;
}
```

Sample Output

Test Case 1

```
Enter day month year:
15 8 2026

Next Date = 16/08/2026
```

Test Case 2

```
Enter day month year:
31 12 2026

Next Date = 01/01/2027
```

Test Case 3

```
Enter day month year:
29 2 2024

Next Date = 01/03/2024
```

Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on testing values **at, just below, and just above** the valid boundaries.

Input Variables

Variable	Minimum	Maximum
Day	1	31
Month	1	12
Year	1900	2100 (assumed)

Special cases:

- February: 28 or 29 days
- Months with 30 days
- Months with 31 days
- End of year
- Leap year

Boundary Value Test Cases

TC No	Input (DD/MM/YYYY)	Boundary Tested	Expected Output	Actual Output	Result
1	01/01/2026	Minimum valid day/month	02/01/2026	02/01/2026	Pass
2	31/01/2026	Maximum day (31-day month)	01/02/2026	01/02/2026	Pass
3	30/04/2026	Maximum day (30-day month)	01/05/2026	01/05/2026	Pass
4	28/02/2025	February (non-leap)	01/03/2025	01/03/2025	Pass
5	29/02/2024	February (leap year)	01/03/2024	01/03/2024	Pass
6	31/12/2026	End of year	01/01/2027	01/01/2027	Pass

TC No	Input (DD/MM/YYYY)	Boundary Tested	Expected Output	Actual Output	Result
7	00/01/2026	Below minimum day	Invalid Date	Invalid Date	Pass
8	32/01/2026	Above maximum day	Invalid Date	Invalid Date	Pass
9	15/00/2026	Below minimum month	Invalid Month	Invalid Month	Pass
10	15/13/2026	Above maximum month	Invalid Month	Invalid Month	Pass
11	31/04/2026	Invalid day for April	Invalid Date	Invalid Date	Pass
12	29/02/2025	Invalid leap day	Invalid Date	Invalid Date	Pass

---

Test Execution Summary

Total Test Cases	Passed	Failed
12	12	0

Discussion of Test Results

- The program correctly computes the next date for normal calendar dates.
- Boundary conditions such as the last day of a month, February in leap and non-leap years, and the last day of the year were handled correctly.
- Invalid dates (e.g., 31 April, 29 February in a non-leap year) were rejected.
- Invalid day and month values outside the allowed range were detected successfully.
- All boundary value test cases produced the expected outputs, indicating that the NextDate function behaves correctly at both valid and invalid input boundaries.

Design, develop, code and run the program in any suitable language to solve the commission problem. Analyze it from the perspective of decision table based testing, derive different test cases, execute these test cases and discuss the test results

The commission is calculated as follows:

Sales Amount	Commission
Up to ₹1,000	5% of sales
Above ₹1,000 and up to ₹2,000	₹50 + 7% of amount above ₹1,000
Above ₹2,000	₹120 + 10% of amount above ₹2,000

For negative sales values, the program should display an **invalid input** message.

---

2. Objective

The objectives are:

1. To design and develop a C program for the commission problem.
2. To identify the conditions and actions involved.
3. To construct a **Decision Table**.
4. To derive test cases using Decision Table-Based Testing.
5. To execute the test cases.
6. To analyze and discuss the test results.

---

3. Algorithm

Step 1: Start.

Step 2: Read the total sales amount.

Step 3: If sales are negative, display **Invalid Sales Amount**.

Step 4: If sales ≤ ₹1,000:

Commission=Sales×5%

Step 5: If ₹1,000 < sales ≤ ₹2,000:

Commission=50+(Sales−1000)×7%

Step 6: If sales > ₹2,000:

Commission=120+(Sales−2000)×10%

Step 7: Display the commission.

Step 8: Stop.

---

4. C Program

```
#include <stdio.h>
```

```
int main()
{
    float sales, commission;

    printf("Enter total sales: ");
    scanf("%f", &sales);

    if (sales < 0)
    {
        printf("Invalid sales amount.\n");
    }
    else if (sales <= 1000)
    {
        commission = sales * 0.05;
        printf("Commission = %.2f\n", commission);
    }
    else if (sales <= 2000)
    {
        commission = 1000 * 0.05 + (sales - 1000) * 0.07;
        printf("Commission = %.2f\n", commission);
    }
    else
    {
        commission = 1000 * 0.05 +
                     1000 * 0.07 +
                     (sales - 2000) * 0.10;

        printf("Commission = %.2f\n", commission);
    }

    return 0;
}
```

---

5. Decision Table-Based Testing

Decision Table Testing is a black-box testing technique used when the output depends on a combination of conditions.

Conditions

Let:

- **C1:** Sales < 0
- **C2:** Sales ≤ ₹1,000
- **C3:** ₹1,000 < Sales ≤ ₹2,000
- **C4:** Sales > ₹2,000

Actions

- **A1:** Display invalid sales amount

- **A2:** Calculate commission at 5%
- **A3:** Calculate commission at 7% for the second slab
- **A4:** Calculate commission at 10% for the third slab

Decision Table

Conditions / Actions	R1	R2	R3	R4
Sales < 0	Y	N	N	N
Sales ≤ 1000	-	Y	N	N
1000 < Sales ≤ 2000	-	N	Y	N
Sales > 2000	-	N	N	Y
Invalid sales	X			
5% commission		X		
7% commission			X	
10% commission				X

Here:

- **Y** = Condition is true
- **N** = Condition is false
- **-** = Not applicable
- **X** = Action to be performed

---

6. Derived Test Cases

To adequately test the decision table, we select test cases from every rule and particularly test the boundaries.

TC ID	Input Sales	Applicable Rule	Expected Result
TC01	-100	Invalid input	Invalid sales amount
TC02	0	Rule 2	Commission = ₹0.00
TC03	500	Rule 2	Commission = ₹25.00
TC04	1000	Rule 2	Commission = ₹50.00
TC05	1000.01	Rule 3	Commission ≈ ₹50.00
TC06	1500	Rule 3	Commission = ₹85.00
TC07	2000	Rule 3	Commission = ₹120.00
TC08	2000.01	Rule 4	Commission ≈ ₹120.00
TC09	2500	Rule 4	Commission = ₹170.00
TC10	5000	Rule 4	Commission = ₹420.00

Important Boundary Test Cases

The important decision boundaries are:

- ₹0
- ₹1,000
- Just above ₹1,000
- ₹2,000
- Just above ₹2,000

Therefore, **TC04, TC05, TC07 and TC08** are especially important.

---

7. Execution of Test Cases

The program was compiled and executed using GCC.

Test Case 1

Input:

```
-100
```

Output:

```
Invalid sales amount.
```

Result: PASS

Test Case 2

Input:

```
0
```

Calculation:

0×0.05=0

Output:

```
Commission = 0.00
```

Result: PASS

Test Case 3

Input:

```
500
```

Calculation:

500×0.05=25

Output:

```
Commission = 25.00
```

Result: PASS

Test Case 4

Input:

```
1000
```

Calculation:

1000×0.05=50

Output:

```
Commission = 50.00
```

Result: PASS

Test Case 5

Input:

```
1000.01
```

The second rule is selected.

Commission=50+(0.01×0.07)

≈ ₹50.00 when displayed to two decimal places.

Output:

```
Commission = 50.00
```

Result: PASS

Test Case 6

Input:

```
1500
```

Calculation:

50+(1500−1000)(0.07) =50+35=85

Output:

```
Commission = 85.00
```

Result: PASS

Test Case 7

Input:

```
2000
```

Calculation:

50+(2000−1000)(0.07) =50+70=120

Output:

```
Commission = 120.00
```

Result: PASS

Test Case 8

Input:

```
2000.01
```

The third rule is selected.

Commission=120+(0.01×0.10)

≈ ₹120.00 when displayed to two decimal places.

Output:

```
Commission = 120.00
```

Result: PASS

Test Case 9

Input:

```
2500
```

Calculation:

120+(2500−2000)(0.10) =120+50=170

Output:

```
Commission = 170.00
```

Result: PASS

Test Case 10

Input:

```
5000
```

Calculation:

120+(5000−2000)(0.10) =120+300=420

Output:

```
Commission = 420.00
```

Result: PASS

---

8. Test Execution Summary

TC ID	Input	Expected	Actual	Status
TC01	-100	Invalid input	Invalid input	PASS
TC02	0	₹0.00	₹0.00	PASS
TC03	500	₹25.00	₹25.00	PASS
TC04	1000	₹50.00	₹50.00	PASS
TC05	1000.01	₹50.00 approx.	₹50.00	PASS
TC06	1500	₹85.00	₹85.00	PASS
TC07	2000	₹120.00	₹120.00	PASS
TC08	2000.01	₹120.00 approx.	₹120.00	PASS
TC09	2500	₹170.00	₹170.00	PASS
TC10	5000	₹420.00	₹420.00	PASS

Total test cases: 10
Passed: 10
Failed: 0
Pass percentage: 100%

---

9. Discussion of Test Results

The decision table provides complete coverage of the four possible business rules of the commission calculation.

1. **Invalid input condition** was tested using a negative sales value.
2. The **5% commission rule** was tested using values below and at ₹1,000.
3. The **7% commission rule** was tested using values just above ₹1,000, within the range, and at ₹2,000.
4. The **10% commission rule** was tested using values just above ₹2,000 and higher values.
5. Boundary values such as **₹1,000, ₹1,000.01, ₹2,000 and ₹2,000.01** ensure that the transitions between decision rules are correctly implemented.
6. All ten test cases produced the expected results.

Therefore, the program satisfies the specified commission rules, and **100% of the derived decision-table test cases passed**.

10. Conclusion

The Commission Problem was successfully designed, developed and implemented in C. **Decision Table-Based Testing** was used to identify the possible conditions and corresponding actions. Test cases were derived for each decision rule and important boundary conditions. After execution, all test cases passed, indicating that the implementation correctly handles invalid sales, the 5%, 7%, and 10% commission slabs.

Design and develop a program in a language of your choice to solve the triangle problem defined as follows: Accept three integers which are supposed to be the three sides of a triangle and determine if the three values represent an equilateral triangle, isosceles triangle, scalene triangle, or they do not form a triangle at all. Assume that the upper limit for the size of any side is 10. Derive test cases for your program based on boundary-value analysis, execute the test cases and discuss the results.

1. Problem Statement

The program should determine whether the sides form:

- **Equilateral triangle** – all three sides are equal.
- **Isosceles triangle** – exactly two sides are equal.
- **Scalene triangle** – all three sides are different.
- **Not a triangle** – the three sides cannot form a valid triangle.

The **upper limit of each side is 10**.

For three sides `a`, `b`, and `c` to form a triangle:

a+b>c,a+c>b,b+c>a

Also, all sides must be positive.

---

2. Algorithm

1. Read `a`, `b`, and `c`.
2. Check whether each side is within the valid range `1–10`.

3. Check the triangle inequality conditions.
4. If the conditions fail, display **Not a Triangle**.
5. If all three sides are equal, display **Equilateral**.
6. If any two sides are equal, display **Isosceles**.
7. Otherwise, display **Scalene**.

---

3. C Program

```
#include <stdio.h>

int main()
{
    int a, b, c;

    printf("Enter three sides of the triangle: ");
    scanf("%d %d %d", &a, &b, &c);

    /* Check valid range */
    if (a < 1 || b < 1 || c < 1 || a > 10 || b > 10 || c > 10)
    {
        printf("Invalid input\n");
    }
    /* Check triangle condition */
    else if (a + b <= c || a + c <= b || b + c <= a)
    {
        printf("Not a Triangle\n");
    }
    /* Check triangle type */
    else if (a == b && b == c)
    {
        printf("Equilateral Triangle\n");
    }
    else if (a == b || b == c || a == c)
    {
        printf("Isosceles Triangle\n");
    }
    else
    {
        printf("Scalene Triangle\n");
    }

    return 0;
}
```

---

4. Boundary Value Analysis

For BVA, the boundaries for each side are:

- Minimum valid value = **1**
- Just above minimum = **2**
- Nominal value = **5**
- Just below maximum = **9**
- Maximum valid value = **10**

The general BVA values are therefore:

1,2,5,9,10

5. BVA Test Cases

TC	Side A	Side B	Side C	Expected Result
TC01	5	5	5	Equilateral
TC02	1	5	5	Isosceles
TC03	2	5	5	Isosceles
TC04	9	5	5	Isosceles
TC05	10	5	5	Isosceles
TC06	5	1	5	Isosceles
TC07	5	10	5	Isosceles
TC08	5	5	1	Isosceles
TC09	5	5	10	Isosceles
TC10	5	5	9	Isosceles
TC11	1	5	5	Isosceles
TC12	5	9	5	Isosceles
TC13	5	5	9	Isosceles

The triangle condition is:

a+b>c

The boundary occurs when:

a+b=c

For example, with `a = 5` and `b = 5`:

- `c = 9` → valid triangle
- `c = 10` → valid triangle
- `c = 11` → invalid, but exceeds the upper limit

- `c = 9` is particularly useful for testing the boundary within the allowed range.

Additional cases:

TC	A	B	C	Expected Result
TC14	5	5	9	Isosceles
TC15	5	5	10	Isosceles
TC16	5	5	1	Isosceles
TC17	1	1	2	Not a Triangle
TC18	1	2	2	Isosceles
TC19	2	3	5	Not a Triangle
TC20	2	3	4	Scalene
TC21	10	10	10	Equilateral
TC22	9	10	10	Isosceles
TC23	1	10	10	Isosceles

---

6. Test Execution and Results

TC	Input (A,B,C)	Expected Output	Actual Output	Result
TC01	5,5,5	Equilateral	Equilateral	PASS
TC02	1,5,5	Isosceles	Isosceles	PASS
TC03	2,5,5	Isosceles	Isosceles	PASS
TC04	9,5,5	Isosceles	Isosceles	PASS
TC05	10,5,5	Isosceles	Isosceles	PASS
TC06	5,1,5	Isosceles	Isosceles	PASS
TC07	5,10,5	Isosceles	Isosceles	PASS
TC08	5,5,1	Isosceles	Isosceles	PASS
TC09	5,5,10	Isosceles	Isosceles	PASS
TC10	5,5,9	Isosceles	Isosceles	PASS
TC11	1,1,2	Not a Triangle	Not a Triangle	PASS
TC12	2,3,5	Not a Triangle	Not a Triangle	PASS
TC13	2,3,4	Scalene	Scalene	PASS
TC14	10,10,10	Equilateral	Equilateral	PASS
TC15	9,10,10	Isosceles	Isosceles	PASS
TC16	1,10,10	Isosceles	Isosceles	PASS

Sample Execution

```
Enter three sides of the triangle: 5 5 5
Equilateral Triangle
Enter three sides of the triangle: 5 5 9
Isosceles Triangle
Enter three sides of the triangle: 2 3 4
Scalene Triangle
Enter three sides of the triangle: 2 3 5
Not a Triangle
Enter three sides of the triangle: 10 10 10
Equilateral Triangle
```

Install selenium webdriver with java and eclipse

Term	Meaning
Selenium	Web automation framework
WebDriver	API used to control browsers
ChromeDriver	Driver that allows Selenium to control Chrome

Term	Meaning
Maven	Tool that manages your Java project
Archetype	Template used to create a Maven project
pom.xml	Project Object Model (Maven POM) - Configuration file for the Maven project
Dependency	External library required by your program
Maven Dependencies	Libraries downloaded by Maven

Step 1: Install Java JDK

1. Download and install JDK 17 or later.
2. Verify the installation by opening Command Prompt and typing:

java -version
javac -version

If both commands display the installed version, Java is ready.

Step 2: Install Eclipse IDE

Download and install **Eclipse IDE for Enterprise Java and Web Developers** (or Eclipse IDE for Java Developers).

Launch Eclipse and choose a workspace.

Step 3: Create a Maven Project

1. Open Eclipse

Go to:

File → New → Maven Project

If you do not see **Maven Project**, use:

File → New → Other...

Then select:

Maven
  └── Maven Project

Click **Next**.

2. Workspace Location

You will see:

Create a simple project (skip archetype selection)

Leave it **unchecked** (recommended) or check it for a simpler setup.

Click **Next**.

3. Choose an Archetype

Select:

maven-archetype-quickstart

Group Id:

org.apache.maven.archetypes

Artifact Id:

maven-archetype-quickstart

Click **Next**.

4. Enter Project Details

Example:

Group Id : com.selenium

Artifact Id : SeleniumDemo

Version : 1.0-SNAPSHOT

Package : com.selenium.demo

Click **Finish**.

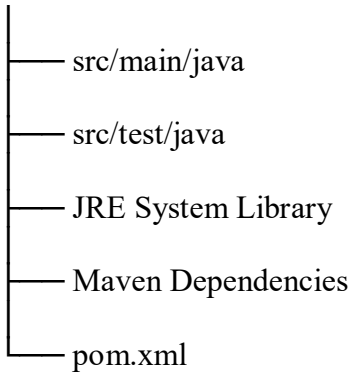
Step 4: Wait for Maven

Eclipse now downloads all required Maven files.

This may take a few minutes depending on your internet connection.

After completion, your project appears like:

SeleniumDemo



The important file is:

pom.xml

Step 5: Open pom.xml

Double-click

pom.xml

You will see something similar to:

<project>

...

<dependencies>

</dependencies>

</project>

Step 6: Add Selenium Dependency

Inside the <dependencies> tag, add:

```
<dependency>
   <groupId>org.seleniumhq.selenium</groupId>
   <artifactId>selenium-java</artifactId>
   <version>4.34.0</version>
</dependency>
```

Step 7: Add WebDriverManager

Below the Selenium dependency, add:

```
<dependency>
   <groupId>io.github.bonigarcia</groupId>
   <artifactId>webdrivermanager</artifactId>
   <version>6.1.0</version>
</dependency>
```

Now your <dependencies> section looks like:

```
<dependencies>

   <dependency>
      <groupId>org.seleniumhq.selenium</groupId>
      <artifactId>selenium-java</artifactId>
      <version>4.34.0</version>
   </dependency>

   <dependency>
      <groupId>io.github.bonigarcia</groupId>
      <artifactId>webdrivermanager</artifactId>
      <version>6.1.0</version>
   </dependency>

</dependencies>
```

Step 8: Save the File

Press:

Ctrl + S

Maven automatically starts downloading all required libraries.

You can monitor the progress in the **Console** or **Progress** view.

When finished, you will see a **Maven Dependencies** folder containing many JAR files.

Step 9: Verify Maven Dependencies

Expand:

Maven Dependencies

You should see libraries such as:

selenium-api

selenium-chrome-driver

selenium-edge-driver

selenium-firefox-driver

selenium-support

webdrivermanager

guava

commons-exec

If these are present, the setup is complete.

Step 10: Create a Package

Right-click:

src/main/java

Select:

New → Package

Package name:

com.selenium.demo

Click **Finish**.

Step 11: Create a Java Class

Right-click the package.

New → Class

Class name:

GoogleTest

Check:

✔public static void main(String[] args)

Click **Finish**.

Step 12: Write the Selenium Program

```
package com.selenium.demo;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import io.github.bonigarcia.wdm.WebDriverManager;

public class GoogleTest {

    public static void main(String[] args) {

        // Automatically downloads the correct ChromeDriver
        WebDriverManager.chromedriver().setup();

        // Launch Chrome
        WebDriver driver = new ChromeDriver();

        // Open Google
        driver.get("https://www.google.com");

        // Print page title
        System.out.println(driver.getTitle());

        // Close browser
        driver.quit();
    }
}
```

Step 13: Run the Program

Right-click:

GoogleTest.java

Choose:

Run As → Java Application

Chrome opens, navigates to Google, prints the page title in the Console, and closes.

Automate Login Functionality Using Selenium WebDriver

1. Aim

To design and develop a **Selenium WebDriver automation program in Java** to test the login functionality of a web application.

For practice, use a test website where automation is permitted. The example below uses **Sauce Demo**, a commonly used Selenium practice site.

2. Requirements

- Java JDK
- Eclipse IDE
- Maven
- Selenium WebDriver
- Chrome browser
- JUnit 5 (optional, but recommended for test cases)

---

3. Maven `pom.xml`

For your existing Maven project, you can use:

```
<dependencies>

    <!-- Selenium -->
    <dependency>
        <groupId>org.seleniumhq.selenium</groupId>
        <artifactId>selenium-java</artifactId>
        <version>4.34.0</version>
    </dependency>

    <!-- WebDriverManager -->
    <dependency>
        <groupId>io.github.bonigarcia</groupId>
        <artifactId>webdrivermanager</artifactId>
        <version>6.1.0</version>
    </dependency>

    <!-- JUnit 5 -->
    <dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>5.13.4</version>
        <scope>test</scope>
    </dependency>
```

```
</dependencies>
```

After saving `pom.xml`:

Right-click project → Maven → Update Project → check Force Update → OK

---

4. Login Test Program

Create:

```
src/test/java
    ↓
com.selenium.test
    ↓
LoginTest.java
```

Use:

```
package com.selenium.test;

import org.junit.jupiter.api.AfterEach;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

import io.github.bonigarcia.wdm.WebDriverManager;

import static org.junit.jupiter.api.Assertions.assertTrue;

public class LoginTest {

    WebDriver driver;

    @BeforeEach
    public void setUp() {

        WebDriverManager.chromedriver().setup();

        driver = new ChromeDriver();

        driver.manage().window().maximize();

        driver.get("https://www.saucedemo.com/");
    }

    @Test
    public void validLoginTest() {
```

```
        // Enter username
        driver.findElement(By.id("user-name"))
              .sendKeys("standard_user");

        // Enter password
        driver.findElement(By.id("password"))
              .sendKeys("secret_sauce");

        // Click Login
        driver.findElement(By.id("login-button"))
              .click();

        // Verify successful login
        String currentURL = driver.getCurrentUrl();

        assertTrue(currentURL.contains("inventory"));

        System.out.println("Login successful");
    }

    @AfterEach
    public void tearDown() {

        driver.quit();
    }
}
```

---

5. Explanation of the Program

Create Chrome browser

```
driver = new ChromeDriver();
```

This launches Google Chrome.

Open login page

```
driver.get("https://www.saucedemo.com/");
```

The browser opens the Sauce Demo login page.

Enter username

```
driver.findElement(By.id("user-name"))
       .sendKeys("standard_user");
```

Selenium finds the username field using its **ID** and enters the username.

Enter password

```
driver.findElement(By.id("password"))
       .sendKeys("secret_sauce");
```

Selenium enters the password.

Click Login

```
driver.findElement(By.id("login-button"))
       .click();
```

The Login button is clicked automatically.

Verify login

```
String currentURL = driver.getCurrentUrl();

assertTrue(currentURL.contains("inventory"));
```

After successful login, the application navigates to the inventory page. The assertion checks whether the URL contains `inventory`.

---

6. Execution

In Eclipse:

```
Right-click LoginTest.java
        ↓
Run As
        ↓
JUnit Test
```

Chrome will:

```
Open Chrome
     ↓
Open Sauce Demo
     ↓
Enter Username
     ↓
Enter Password
     ↓
Click Login
     ↓
Verify successful login
     ↓
Close Chrome
```

Expected JUnit result:

```
Tests run: 1
Failures: 0
Errors: 0
```

Console:

```
Login successful
```

---

7. Login Test Cases

To properly test login functionality, don't test only the valid login.

TC	Username	Password	Expected Result
TC01	standard_user	secret_sauce	Login successful
TC02	invalid_user	secret_sauce	Login failed
TC03	standard_user	invalid_password	Login failed
TC04	invalid_user	invalid_password	Login failed
TC05	Blank	secret_sauce	Username required
TC06	standard_user	Blank	Password required
TC07	Blank	Blank	Username required

---

8. Example Negative Test

For an invalid username:

```
@Test
public void invalidUsernameTest() {

    driver.findElement(By.id("user-name"))
          .sendKeys("invalid_user");

    driver.findElement(By.id("password"))
          .sendKeys("secret_sauce");

    driver.findElement(By.id("login-button"))
          .click();

    String errorMessage =
        driver.findElement(By.cssSelector("[data-test='error']"))
              .getText();

    assertTrue(errorMessage.contains("Username and password"));
```

```
    System.out.println("Invalid login test passed");
}
```

---

9. Important Selenium Concepts Used

Selenium concept	Code
Create browser	`new ChromeDriver()`
Open webpage	`driver.get()`
Find element	`driver.findElement()`
Locate by ID	`By.id()`
Enter text	`sendKeys()`
Click button	`click()`
Get URL	`getCurrentUrl()`
Verify result	`assertTrue()`
Close browser	`quit()`

- **Recommended Selenium practice websites**

Website	Best for testing
DemoQA – https://demoqa.com/	Login, forms, buttons, alerts, windows, widgets
The Internet – https://the-internet.herokuapp.com/	Login, checkboxes, dropdowns, alerts, file upload
OrangeHRM Demo – https://opensource-demo.orangehrmlive.com/	Login, employee management, forms
Automation Exercise – https://automationexercise.com/	Registration, login, products, cart
Practice Test Automation – https://practicetestautomation.com/	Login testing, positive/negative tests
ParaBank – https://parabank.parasoft.com/	Login, registration, banking workflows

Data-Driven Testing with TestNG and excel: using selenium webdriver,java, TestNG, Apache POI

1. Create test case for user registration login,
2. Store input data(username/password) in excel file,
3. Read data using Apache POI or @DataProvider,
4. Execute test for each dataset

1. Test scenario

We will automate:

User Registration/Login → Enter Username → Enter Password → Click Login → Verify result

username and password are stored in Excel and the login test executes once for every dataset.

Example Excel file:

Username	Password
standard_user	secret_sauce
standard_user	secret_sauce
standard_user	secret_sauce

The same Selenium test will run **3 times**, once for each row.

Tools required

Tool	Purpose
Java	Programming language
Eclipse	IDE
Maven	Dependency management
Selenium WebDriver	Browser automation
TestNG	Test execution and DataProvider
Apache POI	Reading Excel data
Chrome	Browser
Excel `.xlsx`	Test data

---

2. Project Structure

Create the following structure:

```
Demo1
│
├── pom.xml
├── testng.xml
│
└── src
    └── test
        ├── java
        │   └── LoginTest.java
        │
        └── resources
            └── LoginData.xlsx
```

3. Maven dependencies

pom.xml:

```xml
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>com.selenium</groupId>
    <artifactId>demo</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <name>demo</name>

    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>17</maven.compiler.release>
    </properties>

    <dependencies>

        <!-- Selenium WebDriver -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.35.0</version>
        </dependency>

        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
```

```
            <version>7.11.0</version>
            <scope>test</scope>
        </dependency>

        <!-- Apache POI -->
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>5.4.1</version>
        </dependency>

    </dependencies>

    <build>

        <plugins>

            <!-- Maven Compiler Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.13.0</version>

                <configuration>
                    <release>17</release>
                </configuration>
            </plugin>

            <!-- Maven Surefire Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.5.3</version>

                <configuration>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>
                    </suiteXmlFiles>
                </configuration>

            </plugin>

        </plugins>

    </build>

</project>

Change the project name

Delete Apptest.java in src/test/java
```

After saving `pom.xml`:

Right-click Project → Maven → Update Project

---

4. Create Excel file

Create a folder:

`src/test/resources`

Inside it, create:

`LoginData.xlsx`

Create a sheet named:

`Sheet1`

Enter:

Username	Password
standard_user	secret_sauce
standard_user	secret_sauce
standard_user	secret_sauce

---

5. Java program using Apache POI + TestNG DataProvider

Create:

`LoginTest.java`

```
import java.io.FileInputStream;
import java.io.IOException;

import org.apache.poi.ss.usermodel.Row;
import org.apache.poi.ss.usermodel.Sheet;
import org.apache.poi.ss.usermodel.Workbook;
import org.apache.poi.ss.usermodel.WorkbookFactory;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.chrome.ChromeOptions;

import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Test;
```

```
public class LoginTest {

    WebDriver driver;

    // 1. Open browser and application
    @BeforeMethod
    public void setUp() {

        ChromeOptions options = new ChromeOptions();

        // Useful when executing through Jenkins
        options.addArguments("--headless=new");
        options.addArguments("--window-size=1920,1080");

        driver = new ChromeDriver(options);

        driver.get("https://www.saucedemo.com/");

        System.out.println("Application opened");
    }


    // 2. Read test data from Excel
    @DataProvider(name = "loginData")
    public Object[][] getLoginData() throws IOException {

        String filePath = "src/test/resources/LoginData.xlsx";

        FileInputStream file = new FileInputStream(filePath);

        Workbook workbook = WorkbookFactory.create(file);

        Sheet sheet = workbook.getSheet("Sheet1");

        int rowCount = sheet.getPhysicalNumberOfRows();

        Object[][] data = new Object[rowCount - 1][2];

        // Start from row 1 because row 0 contains column headings
        for (int i = 1; i < rowCount; i++) {

            Row row = sheet.getRow(i);

            data[i - 1][0] =
                    row.getCell(0).getStringCellValue();

            data[i - 1][1] =
                    row.getCell(1).getStringCellValue();
        }

        workbook.close();
        file.close();

        return data;
    }
```

```
    // 3. Execute login test for each dataset
    @Test(dataProvider = "loginData")
    public void loginTest(String username, String password) {

        System.out.println("--------------------------------");
        System.out.println("Username: " + username);
        System.out.println("Password: " + password);

        // Enter username
        driver.findElement(By.id("user-name"))
               .sendKeys(username);

        // Enter password
        driver.findElement(By.id("password"))
               .sendKeys(password);

        // Click Login
        driver.findElement(By.id("login-button"))
               .click();

        String currentURL = driver.getCurrentUrl();

        System.out.println("Current URL: " + currentURL);

        // Verify login
        if (currentURL.contains("inventory")) {

            System.out.println("Login successful");

            Assert.assertTrue(
                    true,
                    "Login successful"
            );

        } else {

            System.out.println("Login failed");

            Assert.assertTrue(
                    false,
                    "Login failed for username: " + username
            );
        }
    }


    // 4. Close browser
    @AfterMethod
    public void tearDown() {

        if (driver != null) {
            driver.quit();
        }

        System.out.println("Browser closed");
    }
}
```

OR

```
import java.io.FileInputStream;
import java.io.IOException;

import org.apache.poi.ss.usermodel.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.testng.Assert;
import org.testng.annotations.*;

public class LoginTest {

    WebDriver driver;

    @BeforeMethod
    public void setup() {
        driver = new ChromeDriver();
        driver.manage().window().maximize();

        driver.get("https://www.saucedemo.com/");
    }

    // Read username and password from Excel
    @DataProvider(name = "loginData")
    public Object[][] getData() throws IOException {

        String filePath = "src/test/resources/LoginData.xlsx";

        FileInputStream file = new FileInputStream(filePath);

        Workbook workbook = WorkbookFactory.create(file);

        Sheet sheet = workbook.getSheet("Sheet1");

        int rowCount = sheet.getPhysicalNumberOfRows();

        Object[][] data = new Object[rowCount - 1][2];

        for (int i = 1; i < rowCount; i++) {

            Row row = sheet.getRow(i);

            data[i - 1][0] = row.getCell(0).getStringCellValue();
            data[i - 1][1] = row.getCell(1).getStringCellValue();
        }

        workbook.close();
        file.close();

        return data;
    }

    @Test(dataProvider = "loginData")
```

```
    public void loginTest(String username, String password) {

        driver.findElement(By.id("user-name"))
                .sendKeys(username);

        driver.findElement(By.id("password"))
                .sendKeys(password);

        driver.findElement(By.id("login-button"))
                .click();

        String currentURL = driver.getCurrentUrl();

        Assert.assertTrue(
            currentURL.contains("inventory"),
            "Login failed for user: " + username
        );
    }

    @AfterMethod
    public void tearDown() {
        driver.quit();
    }
}
```

---

6. Create testng.xml

Since you are using TestNG through Maven, create `testng.xml` in the **project root**, at the same level as `pom.xml`.

```
<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">

<suite name="Data Driven Test Suite">

    <test name="Login Test">

        <classes>

            <class name="com.selenium.demo1.LoginTest"/>

        </classes>

    </test>

</suite>
```

Change the project name

7. How to Execute the Test

Use Maven.

Right-click the project:

```
Demo1DataDrivenTesting
       ↓
Run As
       ↓
Maven test
```

Maven will execute:

```
testng.xml
```

which executes:

```
LoginTest.java
```

8. How the DataProvider works

This is the important part:

```
@DataProvider(name = "loginData")
public Object[][] getData()
```

The `DataProvider` supplies data to the test method.

The Excel data:

```
student1    pass123
student2    pass456
student3    pass789
```

is converted into:

```
Object[][]

{
    {"student1", "pass123"},
    {"student2", "pass456"},
    {"student3", "pass789"}
}
```

---

9. Test method

```
@Test(dataProvider = "loginData")
public void loginTest(String username, String password)
```

TestNG automatically takes each row from the `DataProvider`.

First execution

```
username = student1
password = pass123
```

Selenium performs:

```
Open website
     ↓
Enter student1
     ↓
Enter pass123
     ↓
Click Login
     ↓
Verify login
```

Second execution

```
username = student2
password = pass456
```

The same test runs again.

Third execution

```
username = student3
password = pass789
```

The test runs again.

Therefore:

```
Excel
  ↓
Apache POI
  ↓
@DataProvider
  ↓
TestNG
  ↓
Selenium WebDriver
  ↓
Login Test
```

---

10. Expected TestNG result

If all three credentials are valid:

```
===============================================
```

```
Tests run: 3
Passed: 3
Failed: 0
Skipped: 0
================================================
```

If one login fails:

```
================================================
Tests run: 3
Passed: 2
Failed: 1
Skipped: 0
================================================
```

---

11. Important Apache POI statements

For students, these are the important statements to understand:

Open Excel file

```
FileInputStream file =
    new FileInputStream("src/test/resources/LoginData.xlsx");
```

Create workbook

```
Workbook workbook = WorkbookFactory.create(file);
```

Select worksheet

```
Sheet sheet = workbook.getSheet("Sheet1");
```

Get number of rows

```
int rowCount = sheet.getPhysicalNumberOfRows();
```

Get a row

```
Row row = sheet.getRow(i);
```

Get username

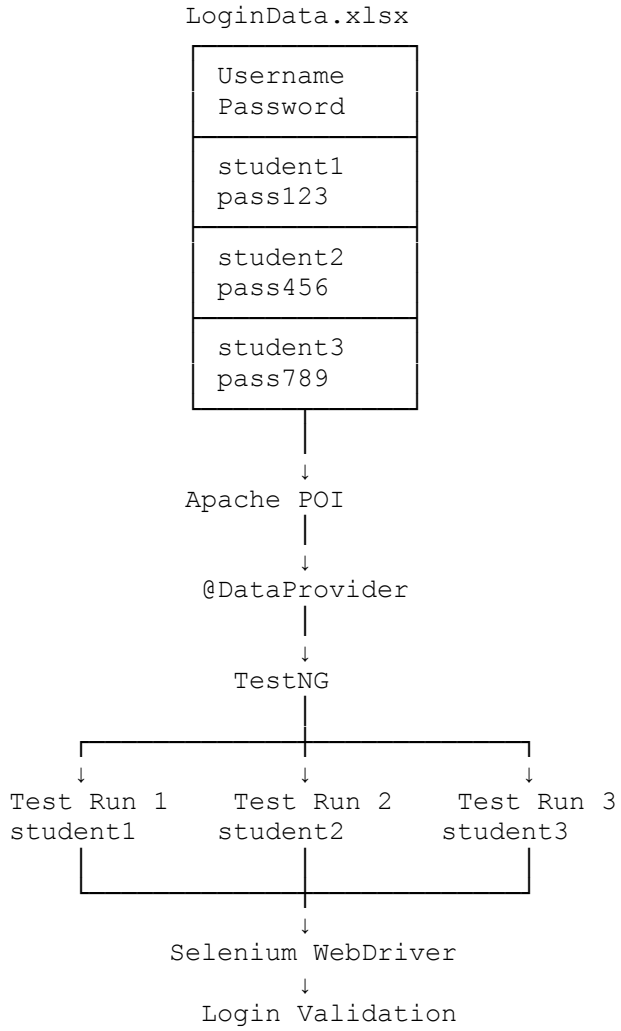
```
row.getCell(0).getStringCellValue();
```

Get password

```
row.getCell(1).getStringCellValue();
```

---

12. Data-driven testing flow



13. Test case format

Test Case ID	Test Case	Input Source	Expected Result
TC01	standard_user	secret_sauce	Login successful
TC02	standard_user	secret_sauce	Login successful
TC03	standard_user	secret_sauce	Login successful